

CodeAnimator

Claude Code Session Report

AWS Infrastructure Discovery, Debugging & End-to-End Fix

April 16, 2026

Session Summary

This document records a complete Claude Code session in which the CodeAnimator repository was analyzed, all AWS cloud infrastructure was pulled down locally, and a broken end-to-end video rendering pipeline was diagnosed and fully repaired through 7 targeted fixes.

1. Project Overview & CLAUDE.md Creation

The session began with an `/init` command to analyze the CodeAnimator codebase and produce a CLAUDE.md file. Claude explored the full repository structure to understand its purpose and architecture before writing any documentation.

What CodeAnimator Does

CodeAnimator transforms Python source code into animated educational MP4 videos via a 3-stage pipeline:

```
Source Code -> AST Parser -> JSON Storyboard -> Manim Renderer -> MP4 Video
```

Repository Structure at Session Start

- poc/ast_service/ - Python AST parsing service with plugin registry pattern
- poc/code_animator_poc/ - Early proof-of-concept Manim animation engine
- backend/fargate-renderer/ - Production Manim renderer (Docker / AWS Fargate)
- backend/lambdaas/ - EMPTY placeholder (code lived only in AWS console)
- backend/step-functions/ - EMPTY placeholder (code lived only in AWS console)
- frontend/ - Empty placeholder, not yet implemented

CLAUDE.md Output

A CLAUDE.md file was created at the repository root. It documents: build and run commands, the three-pillar backend architecture (Dispatcher/Object Registry/Renderer), the 9 visual animation commands, S3 and DynamoDB conventions, and known future work.

2. AWS Infrastructure Discovery

The user confirmed that the real implementation existed only in the AWS console. Claude connected to AWS, fixed the region configuration, and enumerated all resources.

Region Fix

The AWS CLI was misconfigured with region 'eu-east-1', which does not exist, causing every API call to fail. The region was corrected to us-east-1 and authentication was confirmed via `sts:GetCallerIdentity`.

Resources Discovered

Lambda Functions:

CodeAnimator-Parser (Python 3.14) | CodeAnimator-ScriptGenerator (Python 3.14)

Step Function:

CodeAnimator-StepFunction - STANDARD type, JSONata query language

DynamoDB Table:

CodeAnimatorTable - UserID (partition key) + ProjectID (sort key), pay-per-request

S3 Bucket:

code-animator-assets - stores AST JSON, storyboard JSON, and output videos

ECS Cluster:

codeAnimator-EngineCluster

Fargate Task Definitions:

codeAnimator-Engine:1 (2vCPU/16GB)

codeAnimator-EngineV2:1 (2vCPU/4GB) - ACTIVE

codeAnimator-videoRender:1 (1vCPU/4GB) - legacy

ECR Images:

code-animator-engine (active) | code-animator (legacy)

S3 Contents Found

8 prior test projects with `ast_structure.json` and `script.json` files were present, plus one rendered video (`example_simple.mp4`, ~1 MB) confirming the pipeline had worked manually before.

3. Pulling Infrastructure Code Locally

Lambda source code was downloaded via signed S3 URLs (aws lambda get-function). Step Function, DynamoDB schema, and ECS task definitions were saved from the API responses. The repository now contains the full infrastructure as code.

backend/lambda/CodeAnimator-Parser/lambda_function.py:

Parses raw Python code into AST JSON using Python's built-in ast module. Updates DynamoDB status to 'Parsing' and saves ast_structure.json to S3.

backend/lambda/CodeAnimator-ScriptGenerator/lambda_function.py:

Reads the AST from S3 (or directly from the Step Function event), sends it to OpenAI (gpt-5-nano) with a detailed Manim storyboard prompt, saves script.json to S3, updates DynamoDB to 'GeneratingContent'.

backend/step-functions/CodeAnimator-StepFunction.json:

3-step pipeline: Run Parser -> Run ScriptGenerator -> ECS RunTask (Fargate). The Fargate step injects SCRIPT_DATA and PYTHONUNBUFFERED via container overrides.

backend/dynamodb/CodeAnimatorTable.json:

Schema: UserID (PK) + ProjectID (SK, pattern PROJ#{job_id}). Tracks Status, startTime, errorMessage per job.

backend/ecs/task-definitions.json:

All 3 Fargate task definitions with CPU/memory, ECR image URIs, IAM roles, log groups, and notes on which is active.

Security Note

An OPENAI_API_KEY was visible in the Lambda configuration API response. The key was NOT committed. A .env.example template was created instead. The user was advised to rotate the key immediately.

4. Step Function Input Format

The correct JSON payload to trigger the full pipeline is:

```
{
  "UserID":    "some-user-id",
  "ProjectID": "some-project-id",
  "code":      "x = 5\nprint(x)"
}
```

UserID:

DynamoDB partition key. Identifies the user owning the job.

ProjectID:

DynamoDB sort key (stored as PROJ#{ProjectID}) and S3 folder path (projects/{ProjectID}/...). Must be unique per job.

code:

Raw Python source code string. Newlines encoded as \n.

5. End-to-End Test & Root Cause Analysis

A test was run with 'x = 5 / print(x)'. The Step Function reported SUCCEEDED but no video appeared in S3. Full execution history and CloudWatch logs were pulled.

Pipeline Flow Observed

PASS	Parser Lambda - AST saved to S3, DynamoDB -> Parsing
PASS	ScriptGenerator Lambda - script.json saved to S3
FAIL	Fargate - ran ~25 seconds, exited code 0, ZERO log bytes, no video

Root Causes Found (3 bugs)

Bug 1: No CMD/ENTRYPOINT in Dockerfile

Container had nothing to execute. It started, did nothing, exited with code 0. Step Functions treats exit-0 as success even if nothing ran. All CloudWatch log streams showed 0 stored bytes.

Bug 2: main.py ignored SCRIPT_DATA environment variable

Step Functions injects job metadata via SCRIPT_DATA, but main.py only read sys.argv[1] (a local file path). It had no code to read the environment variable, download script.json from S3, or pass it to the renderer.

Bug 3: S3 upload required explicit AWS credentials

main.py checked for AWS_ACCESS_KEY_ID / SECRET / SESSION_TOKEN and silently skipped the upload if any were missing. In Fargate, boto3 uses the task IAM role automatically - no explicit credentials needed or expected.

6. All 7 Fixes Applied

After the three root causes were fixed, five additional issues emerged during Docker builds and runtime - each one revealed only after the previous was resolved.

Fix 1: Dockerfile: Added CMD

```
CMD ["python", "main.py"]
```

Without this the container had no entrypoint and exited immediately with code 0.

Fix 2: main.py: Added fargate_main()

```
Reads SCRIPT_DATA -> parses JSON -> gets script_s3_key + ProjectID
-> downloads script.json from S3 -> writes storyboard to /tmp/
-> calls render_animation() -> uploads to projects/{job_id}/animation.mp4
Entry point auto-routes: if SCRIPT_DATA set -> fargate_main(), else local CLI
```

The container now correctly processes the injected job data.

Fix 3: main.py: Removed explicit AWS credential check

```
Before: checked AWS_ACCESS_KEY_ID/SECRET/TOKEN, silently skipped upload if absent
After:  plain boto3.client('s3') - uses Fargate task IAM role automatically
```

No credentials need to be injected; the role already has S3 write permission.

Fix 4: Dockerfile: Added build-essential

```
apt-get install build-essential
```

manimpango (manim dependency) uses Meson and requires gcc to compile from source.

Fix 5: Dockerfile: Added libjpeg-dev and zlib1g-dev

```
apt-get install libjpeg-dev zlib1g-dev
```

Pillow (manim dependency) compiles from source and needs image codec headers.

Fix 6: requirements.txt + Dockerfile: Pinned setuptools

```
setuptools>=65,<80 (installed as a separate Dockerfile layer)
setuptools >=80 changed how pkg_resources is exposed internally.
```

manim 0.18.0 does 'import pkg_resources' at module load - broken with newer setuptools.

Fix 7: renderer.py: Moved PURE_GREEN alias before its usage

```
# moved to top of file, immediately after imports:
PURE_GREEN = GREEN
```

PURE_GREEN was a class-level default argument value but was only defined 300 lines later. Python evaluates class defaults at definition time, causing NameError.

7. Docker Build Journey

5 build iterations were needed. The apt packages layer took ~8 minutes each time it ran. After Fix 6 was applied, subsequent builds used the cached apt layer and only re-ran the pip install step (~3 minutes).

FAIL

texlive-linux-libertine: package not found on Debian trixie

*Fix: Replaced with texlive-latex-base + texlive-fonts-recommended + cm-super + dvipng***FAIL**

manimpango Meson build failed: no C compiler found

*Fix: Added build-essential to apt packages***FAIL**

Pillow compile failed: missing libjpeg-dev / zlib1g-dev

*Fix: Added libjpeg-dev and zlib1g-dev***FAIL (runtime)**

pkg_resources not found at container startup (setuptools 82.0.1)

*Fix: Pinned setuptools>=65,<80, added separate Dockerfile install layer***SUCCESS**

All packages installed. Image pushed to ECR.

Fix: sha256:3a74dc777afb256c2fa1618b2ff23e0ab8a4ee9a1449a865c7cc9b71e531bc2e

8. Final Verification

After all 7 fixes and the final image push, a new end-to-end test confirmed the complete pipeline works and produces a video in S3.

Test Input

```
{
  "UserID":    "test-user",
  "ProjectID": "debug-test-fixed-004",
  "code":      "x = 5\nprint(x)"
}
```

Step Function Execution Results

PASS	Parser Lambda - ast_structure.json saved to S3
PASS	ScriptGenerator Lambda - script.json saved to S3 (OpenAI storyboard)
PASS	Fargate Container - storyboard downloaded, Manim rendered, animation.mp4 uploaded

S3 Output Verified

```
s3://code-animator-assets/projects/debug-test-fixed-004/
ast_structure.json    624 bytes
script.json          817 bytes
animation.mp4        327,223 bytes  <-- VIDEO CONFIRMED IN S3
```

Pipeline Status: FULLY OPERATIONAL

9. Files Changed in This Session

CREATED	<code>CLAUDE.md</code> <i>Repository documentation for future Claude Code sessions</i>
CREATED	<code>backend/lambda/CodeAnimator-Parser/lambda_function.py</code> <i>Pulled from AWS Lambda console</i>
CREATED	<code>backend/lambda/CodeAnimator-ScriptGenerator/lambda_function.py</code> <i>Pulled from AWS Lambda console</i>
CREATED	<code>backend/lambda/CodeAnimator-ScriptGenerator/.env.example</code> <i>API key template (actual key not committed)</i>
CREATED	<code>backend/step-functions/CodeAnimator-StepFunction.json</code> <i>Pulled from AWS Step Functions console</i>
CREATED	<code>backend/dynamodb/CodeAnimatorTable.json</code> <i>Schema pulled from AWS DynamoDB console</i>
CREATED	<code>backend/ecs/task-definitions.json</code> <i>All 3 task definitions pulled from AWS ECS console</i>
MODIFIED	<code>backend/fargate-renderer/main.py</code> <i>Added <code>fargate_main()</code>, fixed S3 upload (IAM role), auto-detect entrypoint</i>
MODIFIED	<code>backend/fargate-renderer/Dockerfile</code> <i>Added <code>CMD</code>, <code>build-essential</code>, <code>libjpeg-dev</code>, <code>zlib1g-dev</code>, <code>setuptools</code> layer</i>
MODIFIED	<code>backend/fargate-renderer/requirements.txt</code> <i>Added <code>setuptools>=65,<80</code></i>
MODIFIED	<code>backend/fargate-renderer/renderer.py</code> <i>Moved <code>PURE_GREEN</code> alias to top of file (before class that uses it as default arg)</i>